CAMARA
THE TELCO GLOBAL API ALLIANCE

Release Process Update

Maintenance releases and the refactored release automation

Release Management Working Group, October 27, 2026

What changes for you
overview

CAMARA
THE TELCO GLOBAL API ALLIANCE

Unchanged
same commands, same flow

release-plan.yaml | Release Issue | /create-snapshot | /discard-snapshot | /delete-draft | /publish-release

Release PR on the snapshot branch | planned → snapshot-active → draft-ready → published

1 Maintenance releases on a branch

- Patch a released cycle while main moves on
- maintenance/rX, created by the automation
- One Release Issue per line, side by side
- Validation knows the line (P-041, P-042)

2 CHANGELOG per cycle

- Only CHANGELOG/CHANGELOG-rX.md, no root file
- Compare base follows the release type
- Maintenance release: compared to the patched release
- Unchanged APIs are listed as unchanged

3 Clearer feedback, built-in recovery

- Every run writes a step summary
- /sync-issue replies with a result
- Common-file sync PR per line, with comments
- Failed post-release steps: /sync-issue repeats them

4 Under the hood

- Four jobs: plan, snapshot, act, finalize
- Logic in tested Python modules
- Tooling checkout follows the workflow commit
- Fewer cancelled or skipped runs

Status: on the tooling branch release-automation-refactor (tag ra-v1-rc2), tested on ReleaseTest2, then pilot repositories, then main → all repositories via @v1-rc

release-plan.yaml
the baseline for automation

https://github.com/camaraproject/ReleaseManagement/tree/main/documentation/metadata

CAMARA
THE TELCO GLOBAL API ALLIANCE



release-plan.yaml (the intent) new

Where: main or **maintenance/rX.**
Who Edits: Codeowners.
Purpose: tells automation what to do



release-metadata.yaml (the record)

Where: On release tags.
Who Edits: Automation.
Purpose: historical record of release

```yaml
repository:
  release_track: meta-release (or independent)
  meta_release: Sync26
  release_tag: r3.2
  target_release_type: pre-release-rc (or none, …)


dependencies:
  commonalities_release: r3.4
  identity_consent_management_release: r3.3

apis:
  - api_name: my-example-api
    target_api_version: 1.0.0
    target_api_status: rc (or draft, alpha, public)
    main_contacts:
      - @contact1
      - @contact2
```

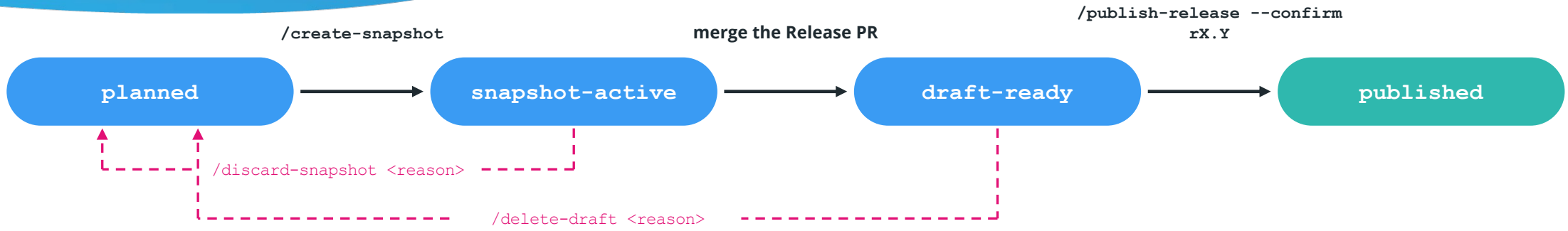
```yaml
repository:
  repository_name: ExampleRepository
  release_tag: r3.2
  release_date: "2026-09-30T10:00:00Z"
  release_type: pre-release-rc
  base_branch: maintenance/r1 new
  src_commit_sha: 7f3a9b2c8e1d4f6a0b5c7e9d3f8a1c4e6b9d2f5a
  release_notes: "Public release of ..."

dependencies:
  commonalities_release: r3.4 (0.6.1)
  identity_consent_management_release: r3.3 (0.4.0)

apis:
  - api_name: my-example-api
    api_version: 1.0.0-rc.2
    api_title: "Quality On Demand"
```

Release flow and commands
on main and on a maintenance line

CAMARA
THE TELCO GLOBAL API ALLIANCE



Release Issue

- created or updated from release-plan.yaml
- on push to main or to the line
- readiness checklist, valid actions

Snapshot + Release PR

- snapshot branch, release-review branch
- CHANGELOG-rX.md section drafted
- review, edit, approve, merge

Draft release

- GitHub draft release with release-metadata.yaml
- versions and files finalised
- publish or delete the draft

Published

- tag rX.Y, source/rX.Y, release/rX.Y
- post-release sync PR(s), codeowner merge
- Release Issue closes

`/sync-issue` (any state, also as workflow dispatch) re-syncs the Release Issue of its line and replies with the result; after a publication whose post-release steps failed, it runs them again and closes the issue.

Maintenance release
from main (A) or from the line (B)

CAMARA
THE TELCO GLOBAL API ALLIANCE

A public release of cycle rX needs a patch
bug fixes only, same APIs

Has main moved on?
release-plan.yaml on main targets a later cycle, a later cycle has already released, or main already holds changes for the next cycle

no

Case A release from main, as usual

- Plan `rX.(N+1)` as `maintenance-release` (or `pre-release-rc` first)
- Same Release Issue on main, same commands
- Works as long as main holds no changes meant for the next cycle
- ReleaseTest2: r1.7 and r1.8 patched r1.6 this way

yes

Case B release from maintenance/rX

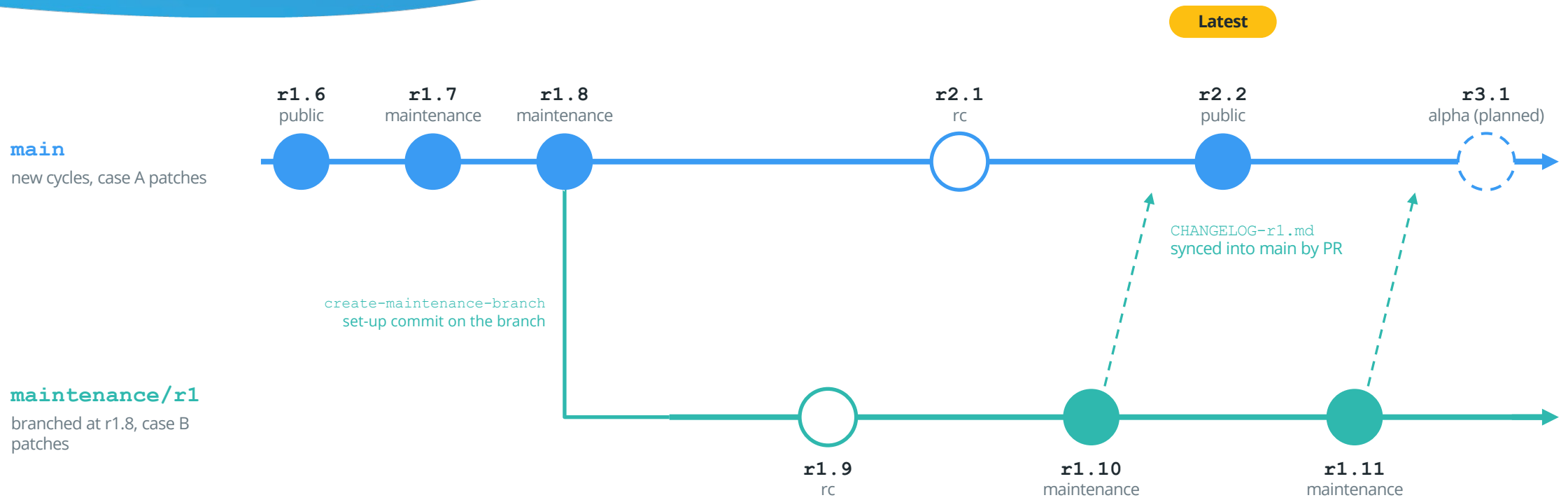
- Create the line once: workflow dispatch `create-maintenance-branch`
- Plan on the line by PR; the merge creates the line's Release Issue
- Same commands on that issue; main keeps planning the next cycle
- ReleaseTest2: r1.9 to r1.11 on maintenance/r1, next to r2.1, r2.2 and r3.1

In both cases the patched release is the newest public or maintenance release of the cycle; the new tag is the cycle's next tag (`P-041`)

- Same API set, patch bumps only; a minor bump is the exception for x ≥ 1 when a later cycle already published a higher major (`P-042`)
- Once `maintenance/rX` exists, main must not plan `rX.*` any more; an rc of the cycle is finished or discarded from main before the branch is created

Two release lines side by side
the ReleaseTest2 example

CAMARA
THE TELCO GLOBAL API ALLIANCE



- Latest stays with r2.2 although r1.10 and r1.11 were published later; the automation restores it if GitHub moved it.
- Each line has its own release-plan.yaml and Release Issue; CHANGELOG compare base: r1.9 and r1.10 against r1.8 (r1.9 is an rc), r1.11 against r1.10.
- r1.7 and r1.8 were released from main (case A) before r2.1 existed.

Case B, step 1
create the maintenance line

Actions › CAMARA Release Automation › Run workflow

Use workflow from

`Branch: main`

action

`create-maintenance-branch`

cycle

`r1`

Run workflow *from main; codeowners only*

1 Preconditions

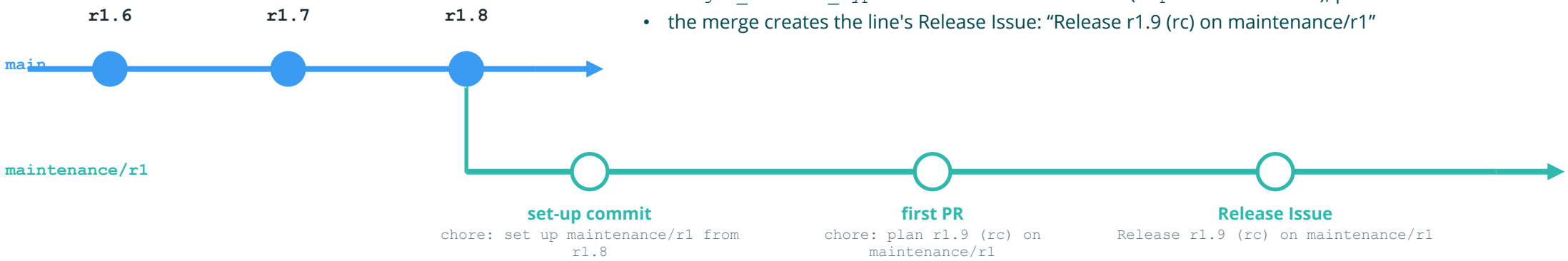
- main's `release-plan.yaml` targets a later cycle (any type, even `none`; a cycle-X tag blocks, also when published)
- the cycle's newest release is public or maintenance, not an rc
- `maintenance/rX` does not exist yet

2 The automation creates the branch

- at the source commit of the patched release (r1.8)
- one set-up commit: caller workflows from main, `CHANGELOG/CHANGELOG-r1.md` of r1.8, `release-plan.yaml` parked at r1.9 with type `none`
- no PR, no Release Issue yet; the step summary names the next step

3 Plan the first release by PR into the branch

- `target_release_type: maintenance-release` (or `pre-release-rc`), patch versions
- the merge creates the line's Release Issue: “Release r1.9 (rc) on maintenance/r1”



Case B, step 2
release and sync on the line

CAMARA
THE TELCO GLOBAL API ALLIANCE

Release r1.11 (maintenance) on maintenance/r1 #147 `release-issue` `release-state:published`

one Release Issue per line; a command on this issue acts on maintenance/r1 only

Plan → **/create-snapshot** → **Draft** → **Published** → **Sync PRs**

- PR into maintenance/r1
- tag `r1.11`, type `maintenance-release`
- merge → Release Issue

- snapshot + Release PR
- `CHANGELOG-r1.md` section, compared to r1.10
- edit, approve, merge

- merge creates the draft release
- `/publish-release --confirm r1.11`

- tag `r1.11`
- `source/r1.11` on the line
- `release/r1.11` pointer branch
- Latest unchanged (r2.2)

- #149 into `maintenance/r1`
- #150 into `main`
- CHANGELOG only, no README change
- issue closes when both exist

Afterwards

- Next fix on the cycle: plan `r1.12` by PR on the line; the line's next release builds on the synced CHANGELOG
- Retire the line: set `target_release_type: none`, close the issue, an admin deletes `maintenance/r1`; `release/r1.Y` branches keep every release reachable
- Branch protection: `maintenance/**` is created and deleted by the automation (admins for retirement); changes go by PR with codeowner review

Validation on a maintenance line
rules that know the line

CAMARA
THE TELCO GLOBAL API ALLIANCE

Rule	What it checks	Where
P-041 plan fits its line	A cycle with a public release gets only maintenance-release or pre-release-rc, with the cycle's next tag On maintenance/rX only rX.*; main must not plan rX.* once the branch exists or a later cycle has released	PRs on main and on the line
P-042 API versions are patches	Same API set as the patched release, no major change, patch bumps only 0.y.z: always patch only; x ≥ 1: a minor bump only if a later cycle published a higher major	PRs on the line (and case A on main)
P-035 published history	Plan stays consistent with the published releases of its own line	scoped to the line
P-033 active release	release-plan.yaml changes are blocked only while a release of the same line is active (snapshot or draft)	a release on the other line does not block

What a codeowner sees on the PR (messages name the branch and the patched release)

```
✗ [P-042] on 'maintenance/r1', which maintains release cycle r1 from r1.10: API 'sample-service' changes '0.1.3' in r1.10 to '0.2.0'; for 0.y.z versions the
minor version is breaking, so a maintenance line allows only patch versions.
Suggestion: Keep '0.1.3' or raise only its patch version for a maintenance release of r1.10; a new minor or major version of 'sample-service' is released from
main in a later release cycle (r2.y).

✗ [P-041] release 'r2.2' is already published in a later cycle, so main can no longer release in cycle r1; maintenance releases of r1 need the branch
'maintenance/r1'.
Suggestion: Create 'maintenance/r1' from main: Actions → CAMARA Release Automation → Run workflow, action create-maintenance-branch, cycle r1. Then plan 'r1.9'
there, or plan a later release on main.
```

CHANGELOG and Latest
what the automation writes

CAMARA
THE TELCO GLOBAL API ALLIANCE

CHANGELOG

- The automation writes one file per cycle: `CHANGELOG/CHANGELOG-rX.md`. An existing root `CHANGELOG.md` stays, but is not maintained by it.
- Candidate changes come from the release's own branch; an API released at its earlier version is listed as unchanged.

Release type	Compared to
alpha	previous release of any type
first rc of a cycle	previous public release; on a maintenance line the patched release
later rc	previous rc of the cycle
public	previous public release
maintenance	patched release (newest public or maintenance release of the cycle)

- "Previous" is ordered by tag version (cycle, then number), not by date.
- After publication the section goes by PR into the line and into `main` (case B), or into `main` with the README block (from main).

Latest on GitHub



`r2.2`
public, main

Latest



`r1.11`
maintenance, published later

not Latest

- A patch of an older cycle never takes Latest from a later cycle's public or maintenance release.
- It takes Latest only while the later cycle has pre-releases alone.
- GitHub marks every published release Latest; the automation sets it back when needed.

Pointers as before

- `source/rX.Y` on the line's branch
- `release/rX.Y` or `pre-release/rX.Y` pointer branch

Better feedback, built-in recovery
and what to do if a step fails

CAMARA
THE TELCO GLOBAL API ALLIANCE

On every run

The step summary says what the run saw and did

Actions › the run › Summary

Trigger	`push to maintenance/r1 (release-plan.yaml)`
Line	`maintenance/r1`
State seen	`planned, Release Issue #147`
Action	`Release Issue synced`

- Answers “why did nothing happen?”: a push that changed nothing, a dispatch on the wrong branch, a precondition that failed
- Fewer noise runs: the bot's own issue writes no longer trigger runs; concurrency at job level

In the Release Issue

Replies follow the real state

- `/sync-issue` answers with a result, no more “Processing”
- the valid-actions list matches the state
- texts on a maintenance line name the line

Common files (Commonalities cache)

- stale cache → one sync PR per line, with a comment when it is created and when the cache is in sync again
- while stale, `/create-snapshot` is blocked and the valid actions point to the sync PR

If a step fails

Release published, but a post-release step failed

- e.g. the sync PR could not be created; the release itself is fine
- the Release Issue stays open and names the retry
- comment `/sync-issue`, or dispatch `sync-issue` on the line's branch: the steps run again and the issue closes

A dispatch was refused

- the run fails and the step summary says which precondition was not met

A command was rejected

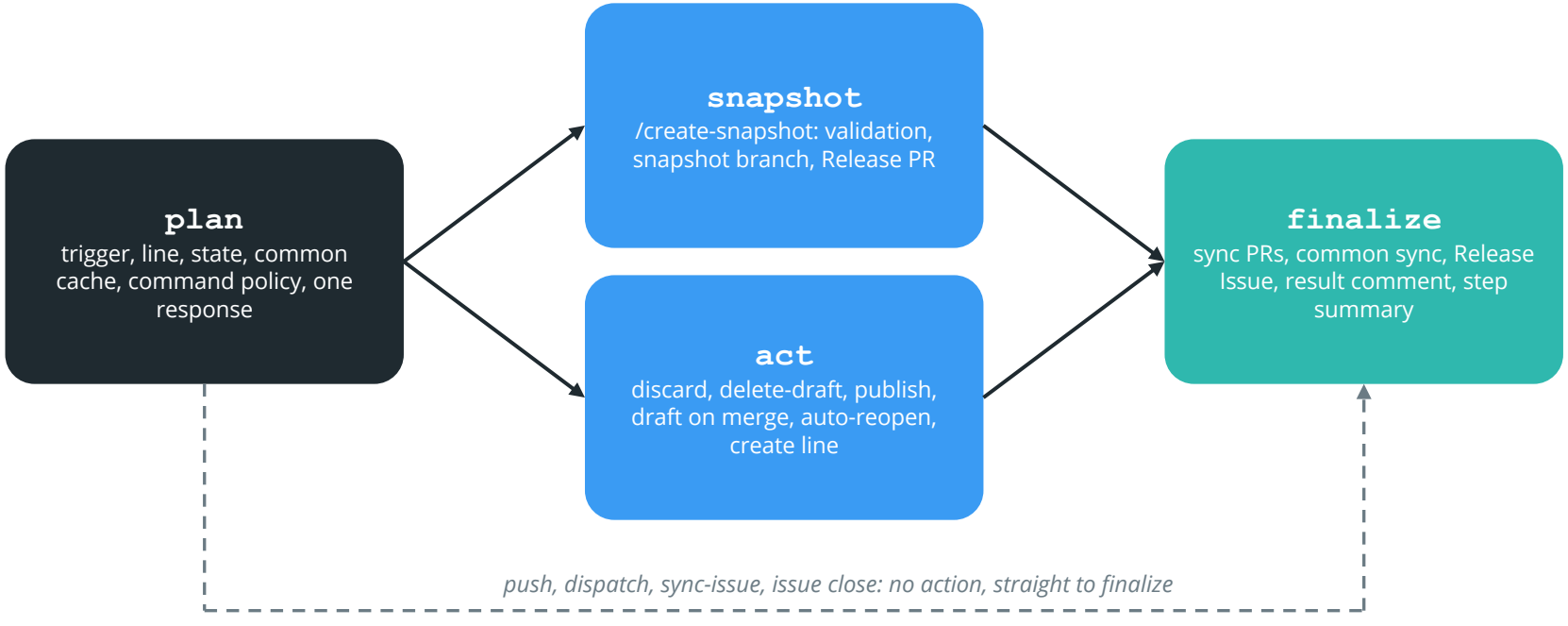
- the reply lists the actions valid in the current state

🔁 **Post-release steps run again — State: published**
Post-release sync PR: #149 and #150 into main (requires codeowner merge). This issue is closed.

bot reply after /sync-issue repaired a publication (ReleaseTest2 #147)

Under the hood
one slide for the curious

CAMARA
THE TELCO GLOBAL API ALLIANCE



Reusable workflow

- four jobs instead of a dozen; logic in tested Python modules, not inline scripts
- the state is derived once (plan) and passed on as a document

Tooling checkout

- follows the exact commit of the called workflow; no override input

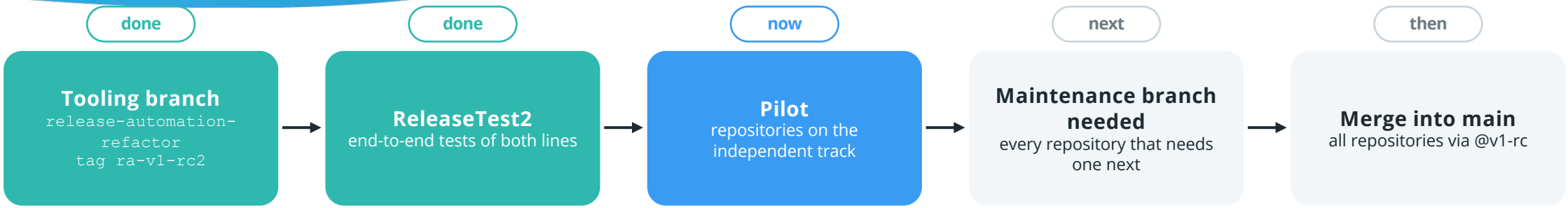
Caller workflow

- push on `maintenance/**`, dispatch actions `sync-issue` and `create-maintenance-branch`
- retired v0 linting workflows removed

Rollout and open points

status as of the presentation date

CAMARA
THE TELCO GLOBAL API ALLIANCE



Not yet decided or in progress

Topic	Where
Publish mode of maintenance releases	ReleaseManagement#675
README release information: describe the release on its tag, not the repository state	ReleaseManagement#730
Which release lines count as supported	ReleaseManagement#459
ReleaseManagement documentation for maintenance releases (maintenance-releases.md)	update pending

Repositories outside the pilot stay on the current @v1-rc automation until the merge into main.

Branches and tags
keep, delete, retire a line

CAMARA
THE TELCO GLOBAL API ALLIANCE

Keep (protected)

- `release/rX.Y` pointer branches of public and maintenance releases
- tags `rX.Y`: never deleted
- `maintenance/rX` while the line is active; created by the automation, changed by PR
- `main`

Delete when done

- `pre-release/rX.Y` after the public release of the cycle
- `release-review/rX.Y-…-preserved`
- `pr-to-main/rX.Y`, `pr-to-maintenance/rX/rX.Y`, `sync-common/…` after their PR merged
- older `source/rX.Y` reference tags

Retire a line

- 1. set `target_release_type: none` on the line (by PR)
- 2. close the line's Release Issue
- 3. an admin deletes `maintenance/rX` (ruleset bypass)
- every release stays reachable through `release/rX.Y` and its tag

Branch protection: `release-snapshot/*` and `maintenance/**` are rulesets with the automation as bypass actor; humans change them by PR only. Recommended clean-up time for intermediary branches: after the public release of the cycle.


Q&A
DEMO